UFR de mathématique et d'informatique
Université de Strasbourg

Université de Strasbourg

UFR de Mathématique et Informatique

Site de Partage d'Images

Licence 3 Informatique

Année universitaire 2025–2026

Informations du projet

Étudiant : Trashi Renaldo

UE : Bases de données

Responsable de l'UE : Gabriel FREY

Enseignant (TP) : Allouche Billal

SGBD : Oracle Database

Langages : SQL, PL/SQL

Objectif du projet : concevoir et implémenter une base de données relationnelle pour un site de partage d'images, en respectant les contraintes fonctionnelles et d'intégrité définies.

Date de rendu : 5 janvier 2026

Table des matières

1 Introduction **3**

1.1 Contexte du projet . . . 3

1.2 Objectifs du projet . . . 3

2 Modèle entité–association **3**

2.1 Diagramme E/A . . . 4

2.2 Justification des cardinalités . . . 4

3 Choix de modélisation et justifications **5**

3.1 Choix des types de données Oracle . . . 5

3.1.1 NUMBER au lieu de INT . . . 5

3.1.2 VARCHAR2(255) pour les mots de passe . . . 5

3.1.3 CLOB pour les descriptions . . . 5

3.1.4 TIMESTAMP vs DATE pour les champs temporels . . . 6

3.1.5 NUMBER(1) avec CHECK pour les booléens . . . 7

3.2 Classification : Catégorie unique vs Labels multiples . . . 7

3.3 Architecture d'archivage . . . 8

3.4 Séquences Oracle pour l'auto-incrémentation . . . 9

3.5 Table Upload_Tracking dédiée . . . 10

3.6 Attribut date_aime dans la relation Aime . . . 10

4 Modèle relationnel **11**

4.1 Résumé des 13 tables . . . 11

4.2 Clés primaires composites vs identifiants artificiels . . . 11

4.3 Décisions sur les suppressions en cascade . . . 12

5 Contraintes d'intégrité **13**

5.1 Contraintes statiques . . . 13

5.1.1 Contraintes CHECK simulant des énumérations . . . 13

5.2 Contraintes dynamiques (déclencheurs) . . . 13

5.2.1 Limitation à 10 albums par utilisateur . . . 13

5.2.2 Anti-spam : 5 images par seconde maximum . . . 14

5.2.3 Prévention du self-liking et self-commenting . . . 15

5.2.4 Auto-initialisation de la date de publication . . . 16

6 Requêtes SQL et justification des choix techniques **16**

6.1 Statistiques utilisateur : Pourquoi des sous-requêtes corrélées ? . . . 16

6.2 Popularité par pays : LEFT JOIN vs INNER JOIN . . . 17

6.3 Index créés pour l'optimisation . . . 18

7 Procédures et fonctions PL/SQL **18**

7.1 Pourquoi utiliser PL/SQL plutôt que SQL pur ? . . . 18

7.2 Export JSON : Pourquoi LISTAGG pour les labels ? . . . 19

7.3 Newsletter : Pourquoi fenêtre de 14 jours pour les likes ? . . . 19

7.4 Recommandation : Algorithme de filtrage collaboratif . . . 20

8 Conclusion **22**

1 Introduction

Ce projet a pour objectif la conception et l'implémentation d'une base de données relationnelle destinée à un site de partage d'images. La base de données constitue un élément central du système, puisqu'elle assure le stockage, la cohérence et l'exploitation des données liées aux utilisateurs, aux images et à leurs interactions.
Le projet s'appuie sur un cahier des charges définissant à la fois les besoins fonctionnels du site et un ensemble de contraintes techniques à respecter, notamment en matière d'intégrité des données, de règles métier et de performances des requêtes.

1.1 Contexte du projet

Un site de partage d'images doit gérer un volume important de données hétérogènes et fortement liées entre elles : utilisateurs inscrits, images publiées, albums, catégories, labels, commentaires et mentions « j'aime ». Ces données doivent être organisées de manière structurée afin d'éviter les redondances et de garantir l'intégrité référentielle.
De plus, certaines fonctionnalités avancées sont attendues, telles que la génération de newsletters personnalisées, l'archivage des images supprimées et la recommandation automatique d'images en fonction de la popularité et des préférences des utilisateurs.

1.2 Objectifs du projet

Les objectifs principaux poursuivis dans ce projet sont les suivants :

- concevoir un modèle entité–association respectant les besoins fonctionnels exprimés dans le sujet ;
- établir un modèle relationnel cohérent à partir du modèle conceptuel ;
- implémenter la base de données en SQL et PL/SQL sous Oracle Database ;
- garantir l'intégrité des données à l'aide de contraintes statiques et dynamiques ;
- permettre l'exécution de requêtes complexes pour l'analyse et la recommandation d'images.

2 Modèle entité–association

Le modèle entité–association (E/A) constitue la première étape de la conception de la base de données. Il permet de représenter de manière abstraite les entités du système, leurs attributs et les relations qui les lient.

2.1 Diagramme E/A

La figure 1 présente le modèle conceptuel de données (MCD) complet du site de partage d'images, réalisé selon les conventions de la notation UML/Merise.

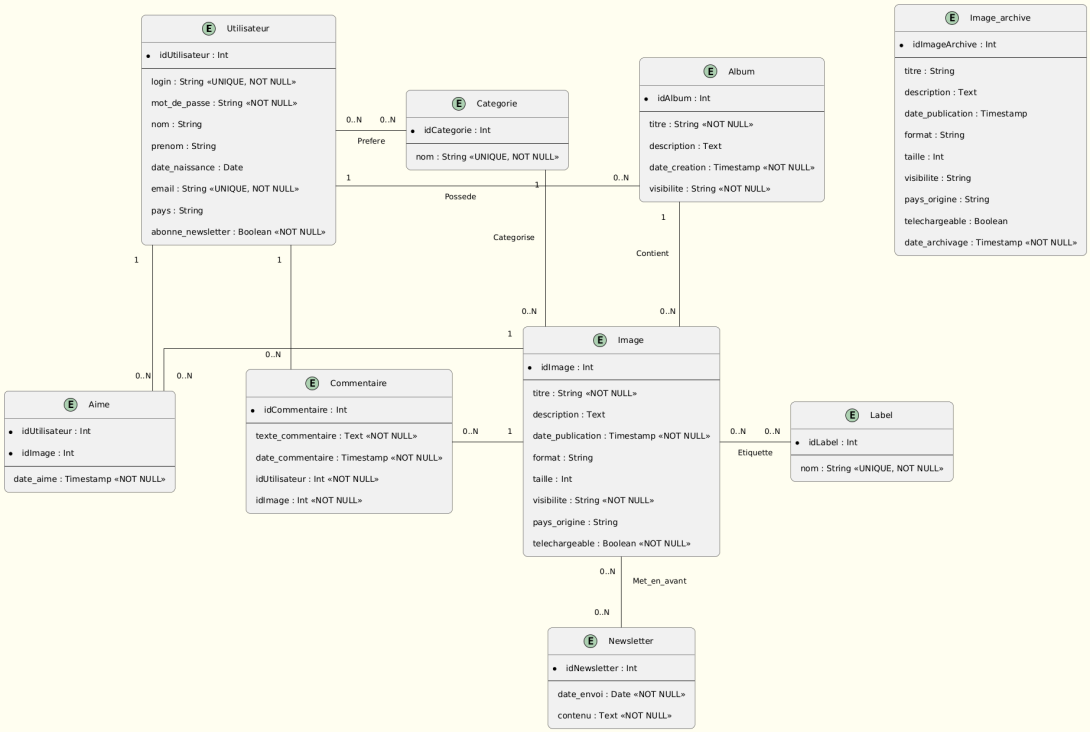

FIGURE 1 – Modèle Conceptuel de Données (MCD)

Ce modèle met en évidence les entités principales du système ainsi que les cardinalités des relations entre elles.

2.2 Justification des cardinalités

Les cardinalités choisies reflètent les règles métier du site :

- **Utilisateur - Album (1,N)** : Cardinalité 1 côté album car chaque album appartient nécessairement à un unique propriétaire, garantissant la responsabilité et le contrôle d'accès. Cardinalité N côté utilisateur car un utilisateur peut organiser ses images dans plusieurs albums thématiques.
- **Album - Image (1,N)** : Une image doit appartenir à exactement un album pour éviter l'ambiguïté sur l'organisation. Le minimum de 0 images par album permet la création d'albums vides qui seront remplis progressivement.
- **Image - Categorie (1,1)** : Choix d'une catégorie unique et obligatoire pour simplifier les statistiques globales et le filtrage. Les labels permettent une classification multi-critères plus fine.
- **Aime, Prefere, Etiquette (N :N)** : Relations plusieurs-à-plusieurs naturelles permettant la flexibilité maximale des interactions utilisateur.

3 Choix de modélisation et justifications

La modélisation de la base de données a nécessité plusieurs décisions importantes, chacune pesant les avantages et inconvénients de différentes approches.

3.1 Choix des types de données Oracle

3.1.1 NUMBER au lieu de INT

Décision : Utilisation systématique du type `NUMBER` pour tous les identifiants.

Justification :

- Oracle ne possède pas de type `INT` natif ; `INT` est un alias pour `NUMBER(38)`
- `NUMBER` offre une précision arbitraire, évitant les risques d'overflow même avec des millions d'enregistrements
- Cohérence avec les pratiques Oracle standard, facilitant la maintenance future

3.1.2 VARCHAR2(255) pour les mots de passe

Décision : Taille de 255 caractères pour le champ `mot_de_passe`.

Justification :

- bcrypt génère des hashes de 60 caractères, SHA-256 produit 64 caractères hexadécimaux
- La taille de 255 permet d'évoluer vers des algorithmes plus robustes (Argon2, scrypt) sans migration de schéma
- Espace pour stocker le préfixe d'algorithme et le sel (format : `$2a$12$saltHashValue...`)

Alternative écartée : VARCHAR2(100) aurait été insuffisant pour certains algorithmes modernes.

3.1.3 CLOB pour les descriptions

Décision : Utilisation du type `CLOB` pour les champs `description` des images et albums.

Justification :

- Les descriptions peuvent être longues (récits de voyage, explications artistiques, contexte historique)
- `VARCHAR2` est limité à 4000 caractères en Oracle (insuffisant pour du texte riche)
- `CLOB` supporte jusqu'à 4 Go de texte, garantissant aucune troncature
- Pas d'impact performance significatif : Oracle stocke les petits CLOB inline (< 4000 caractères)

Alternative écartée : VARCHAR2(4000) aurait tronqué les descriptions longues sans avertissement.

3.1.4 TIMESTAMP vs DATE pour les champs temporels

Décision : Utilisation mixte selon les besoins de précision.

Justification :
TIMESTAMP pour les événements précis :

- `date_publication`, `date_aime`, `date_commentaire` : Précision à la seconde nécessaire
- Déclencheur anti-spam : Calcul d'intervalle < 1s impossible avec DATE
- Tri chronologique précis des interactions (likes, commentaires)
- `date_archivage` : Traçabilité précise des suppressions (audit RGPD)

DATE pour les données à granularité journalière :

- `date_naissance` (Utilisateur) : Précision au jour suffisante, pas besoin d'heure de naissance
- `date_envoi` (Newsletter) : Les newsletters sont envoyées par jour, pas à une heure précise
- Économie d'espace : DATE (7 octets) vs TIMESTAMP (11 octets) pour des millions d'utilisateurs

Avantages du choix :

- Optimisation du stockage sans sacrifier la fonctionnalité
- Sémantique claire : DATE = jour calendaire, TIMESTAMP = moment précis
- Évite la sur-précision inutile (heure de naissance non pertinente)

3.1.5 NUMBER(1) avec CHECK pour les booléens

Décision : Simulation de booléens via `NUMBER(1) CHECK (valeur IN (0,1))`.

Justification :

- Oracle ne supporte pas le type `BOOLEAN` dans les colonnes de table (uniquement en PL/SQL)
- `NUMBER(1)` avec contrainte CHECK garantit l'intégrité au niveau SGBD
- Lisibilité supérieure à `CHAR(1)` qui nécessiterait des conversions constantes
- Compatible avec les langages applicatifs qui utilisent des booléens natifs (Java, Python)

Exemple d'implémentation :

```sql
CREATE TABLE Utilisateur (
    ...
    abonne_newsletter NUMBER(1) DEFAULT 0 NOT NULL,
    CONSTRAINT chk_abonne_newsletter
        CHECK (abonne_newsletter IN (0, 1))
);
```

3.2 Classification : Catégorie unique vs Labels multiples

Décision : Double système de classification : une catégorie obligatoire + labels optionnels.

Justification :

- **Catégorie unique** simplifie les statistiques globales (« combien d'images Nature ? ») et le filtrage principal
- **Labels multiples** permettent une description fine (une photo peut être « Nature + Montagne + Hiver »)
- Évite la prolifération de catégories redondantes qui compliquerait la navigation

Alternative écartée : Catégories multiples auraient rendu ambiguës les requêtes « popularité par catégorie » (une image compterait dans plusieurs catégories).

3.3 Architecture d'archivage

Décision : Table `Image_archive` séparée sans clés étrangères.

Justification :

1. **Performance** : Les requêtes sur images actives (99% du trafic) ne parcourent pas les archives, évitant les scans de tables volumineuses
2. **Indépendance des archives** : Aucune clé étrangère vers `Album` ou `Utilisateur`, permettant la conservation des archives même si le compte d'origine est supprimé (conformité RGPD : traçabilité)
3. **Automatisation** : Déclencheur `BEFORE DELETE` copie automatiquement toutes les métadonnées, aucune intervention manuelle requise

Implémentation du trigger d'archivage :

```
CREATE OR REPLACE TRIGGER trg_archivage_image
BEFORE DELETE ON Image
FOR EACH ROW
BEGIN
    INSERT INTO Image_archive (
        idImageArchive, titre, description,
        date_publication, format, taille,
        visibilite, pays_origine, telechargeable,
        date_archivage
    ) VALUES (
        seq_image_archive.NEXTVAL,
        :OLD.titre, :OLD.description,
        :OLD.date_publication, :OLD.format,
        :OLD.taille, :OLD.visibilite,
        :OLD.pays_origine, :OLD.telechargeable,
        SYSTIMESTAMP
    );
END;
/
```

Coûts acceptés :

- Duplication du schéma (maintenance de deux définitions)
- Pas de nettoyage automatique des archives (nécessite une politique de rétention manuelle)

Alternative écartée : Colonne `archivee BOOLEAN` dans la table `Image` aurait dégradé les performances des requêtes fréquentes.

3.4 Séquences Oracle pour l'auto-incrémentation

Décision : 8 séquences distinctes pour les identifiants.

Justification :

- Oracle ne supporte pas `AUTO_INCREMENT` ; les séquences sont le mécanisme standard
- Garantit l'unicité même en cas d'insertions concurrentes (gestion atomique par Oracle)
- Permet de pré-alloquer des blocs d'identifiants pour améliorer les performances (options `CACHE`, `NOORDER`)

Exemple de création et utilisation :

```sql
-- Creation de sequences
CREATE SEQUENCE seq_utilisateur START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_image START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_image_archive START WITH 1 INCREMENT BY 1;

-- Utilisation lors d'une insertion
INSERT INTO Utilisateur (idUtilisateur, login, email, ...)
VALUES (seq_utilisateur.NEXTVAL, 'alice', 'alice@ex.com', ...);

-- Dans un trigger d'archivage
INSERT INTO Image_archive (idImageArchive, ...)
VALUES (seq_image_archive.NEXTVAL, ...);
```

Alternative écartée : Utiliser une seule séquence globale aurait créé des collisions d'identifiants entre tables différentes.

3.5 Table Upload_Tracking dédiée

Décision : Table séparée pour le suivi anti-spam plutôt qu'ajout de colonnes dans `Utilisateur`.

Justification :

- **Séparation des responsabilités** : La table `Utilisateur` contient des données métier permanentes, `Upload_Tracking` contient des données techniques temporaires
- **Performance** : Les mises à jour fréquentes de `Upload_Tracking` ne déclenchent pas de verrous sur `Utilisateur` (qui pourrait être lu simultanément pour l'authentification)
- **Optionnalité** : Tous les utilisateurs n'auront pas d'entrée dans `Upload_Tracking`, économisant l'espace pour les utilisateurs occasionnels

Coût : Une jointure supplémentaire dans le déclencheur, négligeable face à la clarté architecturale.

3.6 Attribut date_aime dans la relation Aime

Décision : Ajout d'un horodatage `date_aime` dans la table associative.

Justification :

- Permet le calcul de popularité récente (« images populaires des 2 dernières semaines »)
- Facilite l'analyse temporelle des tendances (pics d'activité, saisonnalité)
- Nécessaire pour les newsletters hebdomadaires qui filtrent sur la fraîcheur des likes

Alternative écartée : Sans horodatage, impossible de distinguer un like récent d'un like ancien, rendant les recommandations obsolètes.

4 Modèle relationnel

Le passage du modèle conceptuel au modèle relationnel s'est effectué selon les règles classiques de transformation.

4.1 Résumé des 13 tables

Table	Clé primaire	Clés étrangères
Utilisateur	idUtilisateur	—
Categorie	idCategorie	—
Label	idLabel	—
Album	idAlbum	idUtilisateur
Image	idImage	idAlbum, idCategorie
Image_archive	idImageArchive	— (indépendante)
Newsletter	idNewsletter	—
Aime	(idUtilisateur, idImage)	idUtilisateur, idImage
Commentaire	idCommentaire	idUtilisateur, idImage
Prefere	(idUtilisateur, idCategorie)	idUtilisateur, idCategorie
Etiquette	(idImage, idLabel)	idImage, idLabel
Met_en_avant	(idNewsletter, idImage)	idNewsletter, idImage
Upload_Tracking	idUtilisateur	idUtilisateur

4.2 Clés primaires composites vs identifiants artificiels

Décision : Clés primaires composites pour les tables associatives N :N (`Aime`, `Prefere`, `Etiquette`, `Met_en_avant`).

Justification :

- **Sémantique naturelle** : La combinaison (idUtilisateur, idImage) garantit qu'un utilisateur ne peut liker une image qu'une seule fois
- **Performance** : Oracle crée automatiquement un index sur la clé primaire composite, accélérant les jointures
- **Intégrité** : Évite les doublons sans contrainte UNIQUE supplémentaire
- **Simplicité** : Pas besoin d'un identifiant artificiel supplémentaire (idAime, idPrefere, etc.)

Exception - Commentaire avec idCommentaire :

- Un utilisateur peut commenter plusieurs fois la même image (discussions)
- Nécessite un identifiant unique pour référencer un commentaire spécifique (réponses, modération)
- La PK composite (idUtilisateur, idImage) ne garantirait pas l'unicité

4.3 Décisions sur les suppressions en cascade

Décision : `ON DELETE CASCADE` sur la plupart des clés étrangères.

Justification par cas :

- **Utilisateur → Albums/Images/Likes** : Un compte supprimé doit effacer toutes ses données pour respecter le RGPD (droit à l'oubli)
- **Image → Likes/Commentaires** : Les interactions sur une image supprimée n'ont plus de sens, leur conservation créerait des orphelins
- **Album → Images** : Si l'utilisateur supprime un album, ses images doivent être supprimées (archivage automatique assuré par trigger)

Exemple d'implémentation :

```sql
CREATE TABLE Album (
    idAlbum          NUMBER PRIMARY KEY,
    titre            VARCHAR2(200) NOT NULL,
    description      CLOB,
    visibilite       VARCHAR2(10) NOT NULL,
    idUtilisateur    NUMBER NOT NULL,
    CONSTRAINT fk_album_utilisateur
        FOREIGN KEY (idUtilisateur)
        REFERENCES Utilisateur(idUtilisateur)
        ON DELETE CASCADE,
    CONSTRAINT chk_album_visibilite
        CHECK (visibilite IN ('public', 'prive'))
);

CREATE TABLE Aime (
    idUtilisateur    NUMBER NOT NULL,
    idImage          NUMBER NOT NULL,
    date_aime        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_aime PRIMARY KEY (idUtilisateur, idImage),
    CONSTRAINT fk_aime_image FOREIGN KEY (idImage)
        REFERENCES Image(idImage) ON DELETE CASCADE
);
```

Exception : `Image_archive` n'a aucune FK car les archives doivent persister indéfiniment pour audit et conformité légale.

5 Contraintes d'intégrité

5.1 Contraintes statiques

5.1.1 Contraintes CHECK simulant des énumérations

Décision : `CHECK (visibilite IN ('public', 'prive'))`.

Justification :

- Oracle ne possède pas de type ENUM natif
- La contrainte CHECK garantit l'intégrité au niveau SGBD (impossible d'insérer une valeur invalide)
- Plus maintenable qu'une table de référence pour un petit nombre de valeurs fixes

Alternative écartée : Table `Type_Visibilite` avec FK aurait complexifié inutilement les jointures pour seulement 2 valeurs.

5.2 Contraintes dynamiques (déclencheurs)

5.2.1 Limitation à 10 albums par utilisateur

Décision : Trigger bloquant la création du 11ème album.

Justification :

- Choix du seuil à 10 : Suffisant pour un usage légitime (albums personnels, professionnels, thématiques) tout en bloquant les scripts automatisés qui créeraient des milliers d'albums
- Protection contre l'abus de ressources (chaque album consomme espace disque et mémoire d'indexation)
- Contrainte impossible à exprimer en SQL standard (nécessite comptage dynamique)

Implémentation :

```
CREATE OR REPLACE TRIGGER trg_limite_albums
BEFORE INSERT ON Album
FOR EACH ROW
DECLARE
    v_nb_albums NUMBER;
    v_max_albums CONSTANT NUMBER := 10;
BEGIN
    SELECT COUNT(*) INTO v_nb_albums
```

```
    FROM Album
    WHERE idUtilisateur = :NEW.idUtilisateur;

    IF v_nb_albums >= v_max_albums THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Limite atteinte: Un utilisateur ne peut pas '
            || 'avoir plus de ' || v_max_albums || ' albums.');
    END IF;
END;
/
```

Pourquoi pas en SQL standard ? Une contrainte CHECK ne peut pas référencer d'autres lignes de la table.

5.2.2 Anti-spam : 5 images par seconde maximum

Décision : Trigger calculant l'intervalle entre uploads.

Justification du seuil à 5 :

- Un utilisateur humain ne peut pas sélectionner et uploader 5 images en 1 seconde (temps physique de clic + réseau)
- Bloque les scripts malveillants sans pénaliser l'usage normal (upload par lots de 3-4 photos possible)
- Seuil ajustable facilement en modifiant la constante `v_max_uploads`

Mécanisme :

1. Extraction de l'intervalle en secondes via `EXTRACT(SECOND FROM (SYSTIMESTAMP - derniere_upload))`
2. Si intervalle $<$ 1s et compteur 5 : erreur -20002
3. Si intervalle 1s : réinitialisation du compteur (nouvelle fenêtre temporelle)

Extrait du code :

```
-- Calculer l'intervalle en secondes
v_intervalle := EXTRACT(SECOND FROM
    (SYSTIMESTAMP - v_derniere_upload)) + ...;

IF v_intervalle < 1 THEN
    -- Moins d'une seconde depuis le dernier upload
    IF v_nb_uploads >= v_max_uploads THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Anti-spam: Vous ne pouvez pas ajouter '
            || 'plus de ' || v_max_uploads
            || ' images par seconde.');
    ELSE
        UPDATE Upload_Tracking
```

```
        SET nb_uploads_seconde = v_nb_uploads + 1
        WHERE idUtilisateur = v_idUtilisateur;
    END IF;
ELSE
    -- Reinitialiser le compteur
    UPDATE Upload_Tracking
    SET nb_uploads_seconde = 1
    WHERE idUtilisateur = v_idUtilisateur;
END IF;
```

5.2.3 Prévention du self-liking et self-commenting

Décision : Deux triggers interdisant à un utilisateur d'aimer ou de commenter ses propres images.

Justification commune :

- Évite le biais dans les statistiques de popularité (un utilisateur pourrait artificiellement gonfler ses propres scores)
- Règle métier naturelle sur les réseaux sociaux modernes (Instagram, Facebook interdisent l'auto-like)
- Détection via jointure `Album-Image` pour récupérer le propriétaire
- Les commentaires d'auto-promotion faussent les discussions et réduisent la crédibilité

Implémentation (trigger no_self_like) :

```
CREATE OR REPLACE TRIGGER trg_no_self_like
BEFORE INSERT ON Aime
FOR EACH ROW
DECLARE
    v_proprietaire NUMBER;
BEGIN
    -- Trouver le proprietaire de l'image
    SELECT a.idUtilisateur INTO v_proprietaire
    FROM Album a
    JOIN Image i ON a.idAlbum = i.idAlbum
    WHERE i.idImage = :NEW.idImage;

    IF v_proprietaire = :NEW.idUtilisateur THEN
        RAISE_APPLICATION_ERROR(-20005,
            'Vous ne pouvez pas liker votre propre image.');
    END IF;
END;
/
```

Note : Le trigger `trg_no_self_comment` suit une logique identique pour la table `Commentaire`.

5.2.4 Auto-initialisation de la date de publication

Décision : Trigger définissant automatiquement `date_publication` si non fournie lors de l'insertion d'une image.

Justification :

- Simplifie les insertions manuelles (pas besoin de spécifier SYSTIMESTAMP explicitement à chaque INSERT)
- Garantit qu'aucune image n'a une date de publication NULL, assurant la cohérence des données temporelles
- Permet néanmoins de forcer une date spécifique si nécessaire (utile pour les imports historiques ou migrations)
- Évite les erreurs humaines lors des insertions de test ou de production

Implémentation :

```
CREATE OR REPLACE TRIGGER trg_date_publication_auto
BEFORE INSERT ON Image
FOR EACH ROW
BEGIN
    IF :NEW.date_publication IS NULL THEN
        :NEW.date_publication := SYSTIMESTAMP;
    END IF;
END;
/
```

Alternative écartée : Utiliser DEFAULT CURRENT_TIMESTAMP dans la définition de colonne aurait fonctionné, mais le trigger offre plus de flexibilité pour la logique future (validation de dates, ajustement de fuseaux horaires, etc.).

6 Requêtes SQL et justification des choix techniques

6.1 Statistiques utilisateur : Pourquoi des sous-requêtes corrélées ?

Décision : Utilisation de sous-requêtes plutôt que de JOIN multiples.

Requête complète :

```
SELECT
    u.idUtilisateur, u.login, u.nom, u.prenom,
    (SELECT COUNT(*) FROM Album a
     WHERE a.idUtilisateur = u.idUtilisateur)
```

```sql
     AS nb_albums,
    (SELECT COUNT(*) FROM Image i
     JOIN Album a ON i.idAlbum = a.idAlbum
     WHERE a.idUtilisateur = u.idUtilisateur)
     AS nb_images_publiees,
    (SELECT COUNT(*) FROM Aime ai
     WHERE ai.idUtilisateur = u.idUtilisateur)
     AS likes_donnes,
    (SELECT COUNT(*) FROM Aime ai
     JOIN Image i ON ai.idImage = i.idImage
     JOIN Album a ON i.idAlbum = a.idAlbum
     WHERE a.idUtilisateur = u.idUtilisateur)
     AS likes_recus
FROM Utilisateur u
ORDER BY likes_recus DESC;
```

Justification :

- **Exactitude** : Les JOIN multiples créent un produit cartésien, multipliant les comptages (un utilisateur avec 3 albums et 5 likes donnerait 15 lignes avant GROUP BY)
- **Lisibilité** : Chaque sous-requête compte indépendamment une métrique, évitant les `COUNT(DISTINCT)` complexes
- **Maintenance** : Ajouter une nouvelle statistique nécessite seulement une sous-requête supplémentaire

Coût : Exécution de N sous-requêtes pour N utilisateurs, mais Oracle optimise avec des index sur les FK.

6.2 Popularité par pays : LEFT JOIN vs INNER JOIN

Décision : `LEFT JOIN` entre Categorie et Image.

Requête :

```sql
SELECT
    c.nom AS categorie,
    COUNT(i.idImage) AS nb_images_publiees
FROM Categorie c
LEFT JOIN Image i ON c.idCategorie = i.idCategorie
    AND i.date_publication >= SYSTIMESTAMP - INTERVAL '14' DAY
GROUP BY c.idCategorie, c.nom
ORDER BY nb_images_publiees DESC;
```

Justification :

- Inclut les catégories récemment créées sans images encore (affichage avec compteur à 0)
- Évite la disparition silencieuse de catégories des statistiques
- `INNER JOIN` masquerait l'existence de catégories vides, faussant la perception de la couverture du site

6.3 Index créés pour l'optimisation

Décisions d'indexation :

1. `CREATE INDEX idx_image_date ON Image(date_publication)`

   Justification : Les requêtes de statistiques temporelles filtrent systématiquement sur les 14 derniers jours. Sans index, Oracle scannerait toute la table Image à chaque requête.

2. `CREATE INDEX idx_aime_composite ON Aime(idImage, idUtilisateur)`

   Justification : Index composite accélérant les jointures sur Aime (très fréquentes pour calculer popularité et recommandations). L'ordre (idImage, idUtilisateur) favorise les requêtes « combien de likes pour cette image ? ».

7 Procédures et fonctions PL/SQL

7.1 Pourquoi utiliser PL/SQL plutôt que SQL pur ?

Décision : Implémentation des fonctionnalités avancées en PL/SQL.

Justification :

- **Logique procédurale** : La génération de newsletter nécessite des boucles et concaténations impossibles en SQL déclaratif
- **Gestion d'erreurs** : PL/SQL permet des blocs `EXCEPTION` pour retourner des JSON d'erreur propres
- **Performance** : Exécution côté serveur évitant les allers-retours réseau multiples
- **Réutilisabilité** : Les procédures sont appelables depuis n'importe quelle application (Java, PHP, Python)

7.2 Export JSON : Pourquoi LISTAGG pour les labels ?

Décision : `LISTAGG('"' || l.nom || '"', ', ')` pour agréger les labels.

Justification :

- Agrégation de valeurs multiples en une seule chaîne (évite de retourner N lignes pour N labels)
- Format JSON valide généré directement par Oracle
- Alternative (curseur + boucle) serait plus verbeuse et moins performante

Exemple d'utilisation dans la fonction :

```
-- Recuperer les labels avec LISTAGG
SELECT LISTAGG('"' || l.nom || '"', ', ')
    WITHIN GROUP (ORDER BY l.nom)
INTO v_labels
FROM Etiquette e
JOIN Label l ON e.idLabel = l.idLabel
WHERE e.idImage = p_idImage;

-- Construction du JSON final
v_json := v_json || '  "labels": [' || v_labels || ']';
-- Resultat: "labels": ["Nature", "Montagne", "Hiver"]
```

7.3 Newsletter : Pourquoi fenêtre de 14 jours pour les likes ?

Décision : Comptage des likes sur 14 jours, pas depuis la publication.

Justification :

- Une image populaire publiée il y a 6 mois mais n'ayant plus de likes récents n'est plus « tendance »
- Fenêtre de 14 jours (2 semaines) reflète la popularité actuelle, favorisant le contenu frais
- Encourage les créateurs à publier régulièrement plutôt que de vivre sur d'anciens succès

Requête du curseur (extrait de la procédure) :

```
CURSOR c_images_populaires IS
    SELECT i.idImage, i.titre, c.nom AS categorie,
           u.login AS auteur,
           COUNT(a.idUtilisateur) AS nb_likes
    FROM Image i
```

```sql
    JOIN Album al ON i.idAlbum = al.idAlbum
    JOIN Utilisateur u ON al.idUtilisateur = u.idUtilisateur
    JOIN Categorie c ON i.idCategorie = c.idCategorie
    LEFT JOIN Aime a ON i.idImage = a.idImage
        AND a.date_aime >= SYSTIMESTAMP - INTERVAL '14' DAY
    WHERE i.visibilite = 'public'
        AND i.idCategorie IN (
            SELECT idCategorie FROM Prefere
            WHERE idUtilisateur = p_idUtilisateur
        )
    GROUP BY i.idImage, i.titre, c.nom, u.login
    ORDER BY nb_likes DESC
    FETCH FIRST 20 ROWS ONLY;
```

7.4 Recommandation : Algorithme de filtrage collaboratif

Décision : Système de recommandation basé sur les catégories préférées et la popularité récente.

Principe de l'algorithme :

1. Identifier les catégories préférées de l'utilisateur (table `Prefere`)
2. Sélectionner les images publiques de ces catégories
3. Calculer le score de popularité récente (likes des 14 derniers jours)
4. Exclure les images déjà aimées par l'utilisateur (éviter les redites)
5. Exclure les propres images de l'utilisateur (pas d'auto-promotion)
6. Trier par popularité décroissante et limiter à N suggestions

Justification de la méthode :

- **Filtrage basé sur les préférences** : Exploite les catégories explicitement choisies par l'utilisateur
- **Popularité récente** : Favorise le contenu tendance plutôt que d'anciennes images avec beaucoup de likes historiques
- **Exclusion des images vues** : Optimise l'espace de recommandation (nouveautés uniquement)
- **Scalabilité** : Requête performante grâce aux index sur `idCategorie` et `date_aime`

Alternative écartée : Filtrage collaboratif pur (« utilisateurs similaires ») aurait nécessité des calculs matriciels complexes et coûteux en ressources.

Extrait de la procédure images_conseillees :

```sql
SELECT i.idImage, i.titre, c.nom AS categorie,
       u.login AS auteur,
       COUNT(a.idUtilisateur) AS nb_likes_recents
FROM Image i
JOIN Categorie c ON i.idCategorie = c.idCategorie
JOIN Album al ON i.idAlbum = al.idAlbum
JOIN Utilisateur u ON al.idUtilisateur = u.idUtilisateur
LEFT JOIN Aime a ON i.idImage = a.idImage
    AND a.date_aime >= SYSTIMESTAMP - INTERVAL '14' DAY
WHERE
    -- Categories preferees de l'utilisateur
    i.idCategorie IN (
        SELECT idCategorie FROM Prefere
        WHERE idUtilisateur = p_idUtilisateur
    )
    -- Exclure les images deja likees
    AND i.idImage NOT IN (
        SELECT idImage FROM Aime
        WHERE idUtilisateur = p_idUtilisateur
    )
    -- Images publiques uniquement
    AND i.visibilite = 'public'
    -- Exclure ses propres images
    AND al.idUtilisateur != p_idUtilisateur
GROUP BY i.idImage, i.titre, c.nom, u.login
ORDER BY nb_likes_recents DESC
FETCH FIRST p_nb_suggestions ROWS ONLY;
```

8 Conclusion

Ce projet a permis de concevoir et d'implémenter une base de données relationnelle complète dédiée à un site de partage d'images, en respectant les exigences fonctionnelles et techniques du cahier des charges.

Réalisations principales :

- 13 tables interconnectées assurant l'intégrité référentielle
- 8 séquences Oracle pour la gestion des identifiants auto-générés
- 6 déclencheurs PL/SQL implémentant les règles métier complexes
- 5 requêtes SQL optimisées avec index
- 3 procédures/fonctions PL/SQL pour les fonctionnalités avancées

Décisions architecturales majeures :
Chaque choix technique a été justifié par l'analyse des alternatives et des compromis. La séparation des images actives et archivées favorise les performances au prix d'une maintenance accrue. L'utilisation de types Oracle natifs (NUMBER, TIMESTAMP) plutôt que d'alias garantit la portabilité et évite les pièges de conversion. Les déclencheurs PL/SQL implémentent des contraintes métier impossibles à exprimer en SQL déclaratif.

La base de données réalisée constitue une fondation solide et évolutive, pouvant être enrichie ultérieurement (système de notifications, gestion de versions d'images, statistiques avancées) ou intégrée à une application web complète.

Remerciements

Je remercie M. Gabriel FREY, responsable de l'UE, ainsi que M. Allouche Billal, pour l'encadrement, les consignes fournies et les connaissances transmises tout au long de ce projet.

Fin du rapport